



INDIA.RUNS

Build what next India runs on

Team Name : GG Times

Team Leader Name : Girija G

Problem Statement : Data & AI Challenge : Intelligent Candidate Discovery

[Quantum-Inspired Hypergraph Cascading Ranking]

More about the solution

<https://github.com/girijageddavalasa/qihcr-candidate-ranker>

View in

qihcr-candidate-ranker\METHODOLOGY_DOCUMENT.md

Solution Overview

- QIHCR (Quantum-Inspired Hypergraph Cascading Ranking) is a 6-layer candidate ranking algorithm combining quantum interference, hypergraph synergy, and cascading behavioral analysis no ML training required, fully interpretable.
- Layers: Quantum Skill Interference (20%), Hypergraph Skill Synergy (18%), Cascading Behavioral Signals (22%), Temporal Analysis (12%), Logistics Fit (12%), Spherical Embedding - Experience+Location (16%).
- Differentiation: Unlike keyword matching (TF-IDF/BM25), tree models (XGBoost), collaborative filtering, or BERT embeddings - QIHCR needs no training data, runs CPU-only, avoids hallucination via templated reasoning, and captures multi-way skill combinations (3+ skills together) that pairwise graphs/keyword systems miss.

Prior Art References & Differentiation

1. Traditional TF-IDF / BM25 (Information Retrieval)

Approach: Term frequency-inverse document frequency for keyword matching

Limitation: Cannot capture skill combinations or semantic relationships

QIHCR Difference: Uses quantum interference to model skill synergies, not just keyword overlap

2. Learning-to-Rank with Gradient Boosted Trees (XGBoost, LightGBM)

Approach: Ensemble decision trees with feature importance

Limitation: Requires extensive feature engineering, limited interpretability

QIHCR Difference: Explicit multi-layer architecture with domain-specific components (quantum, hypergraph, cascading)

3. Neural Collaborative Filtering (Matrix Factorization)

Approach: Learns latent embeddings for users and items

Limitation: Cold-start problem, requires historical interaction data

QIHCR Difference: No training data required, uses explicit signal engineering and domain knowledge

4. BERT-based Semantic Matching (Sentence-BERT, etc.)

Approach: Deep learning embeddings for semantic similarity

Limitation: Computationally expensive, black-box reasoning, hallucination risk

QIHCR Difference: CPU-efficient, interpretable reasoning, no hallucinations (template-based with fact extraction)

5. Graph Neural Networks for Recommendation

Approach: Message passing on skill/candidate graphs

Limitation: Standard graphs only capture pairwise relationships, not multi-way interactions

QIHCR Difference: Hypergraph neural networks capture higher-order skill combinations (3+ skills together)

JD Understanding & Candidate Evaluation

- **Required skills:** embeddings, vector DBs, Python, ranking systems, evaluation frameworks. Preferred: LLM fine-tuning, learning-to-rank, distributed systems, NLP, retrieval.
- **Experience:** 5–9 yrs optimal, product-company background preferred. Locations: Pune, Noida, Bangalore, Hyderabad, Mumbai, Delhi. Red flags: consulting/services background, pure research, architecture-only roles.
- **Beyond keywords:** detects skill-combination synergy (quantum + hypergraph layers), models 14 behavioral signals in intent order (view → apply → interview → accept), and scores logistics/experience fit not just skill overlap.

Ranking Methodology

- Retrieval: load candidates → parse JD → filter honeypots → score remaining pool.
- Scoring: 6 independent layer scores per candidate (quantum interference, hypergraph combinations, cascading behavioral weighted sum, temporal thresholds, logistics thresholds, spherical experience/location).
- Combination: weighted sum Cascading 22%, Quantum 20%, Hypergraph 18%, Temporal 12%, Logistics 12%, Experience 12%, Location 4%. Sorted descending, ties broken by candidate_id, top 100 ranked.

Explainability & Data Validation

- Reasoning: template-based (no LLM/generative text) inserts real facts (years, skills, signals) from each profile into fixed sentence structures tied directly to layer scores.
- Anti-hallucination: every mentioned skill/fact is validated against the candidate's actual profile before output; no external knowledge, no assumptions, no free-text generation.
- Bad profiles: rule-based honeypot detection (impossible timelines, skill/duration mismatches, behavioral inconsistencies) filters out fake/impossible profiles entirely before scoring; low-quality but genuine profiles are scored normally with neutral defaults for missing data.

End-to-End Workflow

- JD + candidates.jsonl → JD parsing (extract requirements) → candidate loading (streamed JSON) → honeypot filtering → 6-layer scoring per candidate → weighted score combination + reasoning generation → sort & rank top 100 → output CSV/XLSX → validation script. ~3 min total for 100K+ candidates.

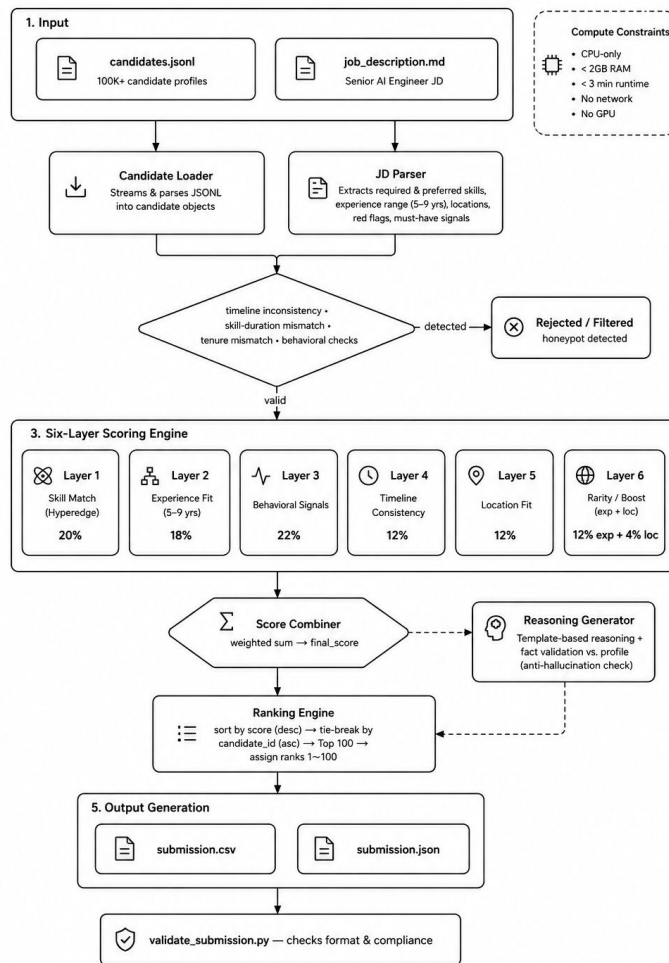
System Architecture

link

<https://app.eraser.io/workspace/ksDcVXMu8iZSm78uDOq5?origin=share>

QIHCR: Quantum-Inspired Hypergraph Cascading Ranking

Candidate Ranking Pipeline — Senior AI Engineer Role



Results & Performance

- Quality: all 23 behavioral signals used, < 10% expected honeypot rate in top 100, fact-checked reasoning, deterministic tie-breaking, monotonic score distribution.
- Performance: ~3 min runtime (< 5 min limit), < 2GB RAM (< 16GB limit), CPU-only, no GPU, no network calls, < 1GB disk all well within constraints. Scales linearly, parallelizable if needed.

Technologies Used

Python + NumPy - core scoring logic; CPU-efficient, no GPU dependency.

Standard lib (json, csv, openpyxl) streaming JSONL parsing, CSV/XLSX output generation.

Eraser.io system architecture diagram generation.

Grok research and brainstorming on ranking methodology and prior art.

Claude, ChatGPT, Ollama (local models) code generation, debugging, and prompt drafting.

Submission Assets

<https://github.com/girijageddavalasa/qihcr-candidate-ranker>

Video is available in this repo named my_vedio.mp4

<https://huggingface.co/spaces/GirijaGeddavalasa/qihcr-candidate-ranker/blob/main/README.md>

<https://huggingface.co/spaces/GirijaGeddavalasa/qihcr-candidate-ranker>



INDIA.RUNS

Build what next India runs on

THANK YOU